Investigating with commands

NUnix · L3 MIAGE · CM2 · 90 minutes

ISTIC · University of Rennes 2026–2027



Louis Ledoux

Investigating with commands

Université de Rennes
Louis Ledoux

A computer is slow. Would adding RAM help?
Check which resource is under pressure first.

1. Check CPU, memory and disk usage.
2. Find large files and inspect application logs.
3. Count error codes with a pipeline.

1. Processes and resources

Which programs are running, and where are the resources going?

CPU, memory and disk space

Université de Rennes
Louis Ledoux

Resource	Question	First observation
CPU	Is computation keeping a processor busy?	`top`, `ps`
RAM	Is memory pressure affecting the system?	`free -h`
Disk	Can the filesystem store more data?	`df -h .`

Adding RAM does not create free space on a full disk.

Running in the foreground

Université de Rennes
Louis Ledoux

A **process** is a running instance of a program.

```
sleep 10
```

The shell waits for this foreground command to finish before showing a new prompt.

Ctrl+C requests an interrupt of the foreground job.

Inspecting processes

Université de Rennes
Louis Ledoux

```
ps
ps -ef
```

Field	Meaning
PID	Process identifier
TTY	Associated terminal, if any
TIME	Accumulated CPU time
CMD	Command

Checking memory usage

Université de Rennes
Louis Ledoux

```
free -h
ps -eo pid,comm,%cpu,%mem --sort=-%mem
```

- Read **available** memory; cache use is normal.
- Identify the application's process and follow it over time.
- Check logs before attributing every delay to memory.

Checking disk usage

Université de Rennes
Louis Ledoux

Tool	Scope	Question
`df`	A filesystem	How much space is available?
`du`	A directory tree	Which directories use space?
`find`	Individual entries	Which files fit my criteria?

Start broad, then inspect a **specific file**.

Find which directories use space

Université de Rennes
Louis Ledoux

```
df -h "$HOME"
du -h --max-depth=1 "$HOME"
```

`df` shows usage of the filesystem containing your home.
`du` shows how much space your directories use.

Finding files

Université de Rennes
Louis Ledoux

```
find . -type f -name '*.txt'
```

- `.` is the **starting directory**.
- `find` also visits its subdirectories.
- `-type f` selects regular files.
- `-name '*.txt'` tests each entry's name.

The quotes let **find** receive the pattern unchanged.

Search criteria

Université de Rennes
Louis Ledoux

Criterion	Matches
`-iname '*.txt'`	Name, ignoring case
`-user alice`	Entries owned by Alice
`-type d` / `-type l`	Directories / symbolic links
`-size +100M`	Size greater than 100 MiB
`-mtime -2`	Modified less than 48 hours ago

```
find . -type f -size +100M
```

Checkpoint: find

Université de Rennes
Louis Ledoux

1. Find regular `.c` files below the current directory.
2. Why quote `'*.c'`?

1. `find . -type f -name '*.c'`
2. So the shell passes the pattern to `find` without expanding it first.

2. Searching text

How can I find the relevant lines in a large log?

A small log file

Université de Rennes
Louis Ledoux

Save these four lines as `app.log`:

```
10:00 INFO STARTED
10:01 ERROR DISK_FULL
10:02 ERROR TIMEOUT
10:03 ERROR DISK_FULL
```

Field	Meaning
1	Time
2	Severity
3	Event code

Finding text with grep

Université de Rennes
Louis Ledoux

```
grep ' ERROR ' app.log
grep -r ' ERROR ' .
grep -n 'DISK_FULL' app.log
```

- Print **matching lines** from a file.
- `-r` searches directories recursively.
- `-n` includes line numbers.

Exit status: **0** = match, **1** = no match, **2** = error in GNU `grep`.

Where the name grep comes from

Université de Rennes
Louis Ledoux

The editor ed had the command pattern:

```
g/re/p
```

- **g**: apply globally to matching lines.
- **re**: a regular expression.
- **p**: print those lines.

`grep` makes that text search available as a separate command.

Patterns in grep

Université de Rennes
Louis Ledoux

Regular expression	Meaning
`.`	Any one character
`m*`	Zero or more repetitions of `m`
`[1-3]`	A digit from 1 to 3

```
grep 'm.*t' names.txt
grep 'toto[1-3]' names.txt
```

In a regular expression, * repeats the **preceding item**.

Who interprets this pattern?

Université de Rennes
Louis Ledoux

```
printf '%s\n' *.log
grep 'DISK.*' app.log
find . -name '*.log'
```

1. The **shell** expands a filename pattern.
2. **grep** interprets a regular expression.
3. **find** interprets a quoted filename pattern.

Input, output and status

Université de Rennes
Louis Ledoux

Channel	Number	Usual terminal behaviour
`stdin`	**0**	Read keyboard input
`stdout`	**1**	Display normal output
`stderr`	**2**	Display diagnostic messages

The **exit status** is a number reported when a command finishes. It is separate from the text it prints.

Exit status

Université de Rennes
Louis Ledoux

```
true
echo "$?"
false
echo "$?"
```

0 means success / true. **Nonzero** means failure / false.
`$?` is the status of the **most recent command**.

3. Redirections and pipes

How can I connect commands and save their results?

Reading input from a file

Université de Rennes
Louis Ledoux

```
sort < names.txt
```

The shell opens `names.txt` as **standard input** for `sort`.

`sort` writes the sorted lines to standard output, which still points to the terminal.

Saving output

Université de Rennes
Louis Ledoux

```
sort names.txt > sorted.txt
echo 'Zoé' >> sorted.txt
```

Operator	Effect on the destination file
>	Create it, or **replace** its contents
>>	Create it, or **append** to its contents

Redirecting errors

Université de Rennes
Louis Ledoux

```
find . -name '*.txt' > matches.txt 2> errors.txt
```

- > redirects **stdout**, descriptor 1.
- 2> redirects **stderr**, descriptor 2.
- Normal results and diagnostics go to separate files.

Redirections are applied from left to right

Université de Rennes
Louis Ledoux

```
ls notes.txt missing > all.txt 2>&1
ls notes.txt missing 2>&1 > out.txt
```

Command	stdout ends up…	stderr ends up…
First	In `all.txt`	In `all.txt`
Second	In `out.txt`	At the terminal

2>&1 copies stdout's destination **at that moment**.
It does not make stderr follow later changes to stdout.

Why did the file become empty?

Université de Rennes
Louis Ledoux

```
sort names.txt > names.txt
```

The shell opens and truncates `names.txt` **before** starting `sort`. The input is already gone.

Write to a **different file**, then inspect the result.

A pipe between two commands

Université de Rennes
Louis Ledoux

```
grep ' ERROR ' app.log | wc -l
```

`grep` writes matching lines to its standard output.
The pipe connects that output to `wc`'s standard input.

The result is the **number of matching lines**.

Connecting programs like garden hoses

Université de Rennes
Louis Ledoux

Doug McIlroy described connecting programs so that one program's output could become another's input.

```
grep ' ERROR ' app.log | wc -l
```

Each tool has a small job.
The shared text stream lets us combine them.

1. Select errors and extract their codes

Université de Rennes
Louis Ledoux

```
grep ' ERROR ' app.log |
    cut -d ' ' -f 3
```

```
DISK_FULL
TIMEOUT
DISK_FULL
```

2. Count each error code

Université de Rennes
Louis Ledoux

```
grep ' ERROR ' app.log |
    cut -d ' ' -f 3 |
    sort | uniq -c | sort -nr
```

```
      2 DISK_FULL
      1 TIMEOUT
```

`uniq -c` counts **adjacent** equal lines. Sort first.

Save and display output with tee

Université de Rennes
Louis Ledoux

```
sort names.txt | tee sorted.txt
```

tee copies its input to a file **and** standard output.

Think of a T-shaped pipe fitting: the stream has two outlets.

Combining arguments and redirections

Université de Rennes
Louis Ledoux

```
sort < names.txt > sorted.txt
sort < names.txt | head -n 3 > first-three.txt
```

- `< names.txt` selects the input stream.
- `-n 3` remains an **argument** to head.
- `> first-three.txt` selects the output file.

Checkpoint: redirections

Université de Rennes
Louis Ledoux

1. Which operator appends to a file?
2. What does `a | b` connect?
3. Does `a > out.txt` also redirect standard error?

1. `>>`
2. a's stdout to b's stdin.
3. No. Use `2>` to redirect stderr.

The exit status of a pipeline

Université de Rennes
Louis Ledoux

```
false | sort
echo "$?"
set -o pipefail
false | sort
echo "$?"
```

Output: **0**, then **1**.
Bash normally reports the last pipeline command’s status.

CM2 summary: commands and pipelines

Université de Rennes
Louis Ledoux

- Check **CPU**, **memory** and **disk** usage.
- Find files and filter lines with `grep`.
- Combine commands with pipes and redirections.

Optional reference

More search expressions and interactive process control

Combining criteria

Université de Rennes
Louis Ledoux

```
find /usr -name '*.c'
find . -user alice -name '*.c'
find . -not \( -name '*.obj' -o -name '*.map' \)
```

- Adjacent tests use **AND**. `-a` makes it explicit.
- `-o` means **OR**. `-not` negates a test.
- Escaped parentheses group tests for `find`.

Running a command on matches

Université de Rennes
Louis Ledoux

```
find . -type f -name '*.txt' -exec wc -l {} \;
```

- `-exec` starts the command to run.
- `{}` stands for the current matching path.
- `\;` ends the command without ending the shell's command line.

Running in the background

Université de Rennes
Louis Ledoux

```
sleep 60 &
jobs
fg %1
```

- & lets the shell return to the prompt immediately.
- `jobs` lists jobs managed by this shell.
- `fg %1` brings job **1** to the foreground.

Suspending and resuming a job

Université de Rennes
Louis Ledoux

Action	Effect
Ctrl+Z	Suspend the foreground job
`bg %1`	Resume job 1 in the background
`fg %1`	Resume job 1 in the foreground

A **job number** belongs to a shell.
A **PID** identifies a process in the system.

Sequential and asynchronous execution

Université de Rennes
Louis Ledoux

```
who | sort; date
sleep 10 &
date
```

- ; runs the next command after the previous one finishes.
- & starts work asynchronously.
- The final `date` can run while `sleep` is still active.

Signals and end of input

Université de Rennes
Louis Ledoux

Action	Meaning
`kill PID`	Send `SIGTERM` to a process
Ctrl+C	Send `SIGINT` to the foreground process group
`Ctrl+\`	Send `SIGQUIT` to the foreground process group
Ctrl+Z	Send `SIGTSTP` to suspend the foreground group
Ctrl+D	Terminal end-of-input action